

How the Simulated Customer Is Generated

A trace of the JSONL → customer-message pipeline in `evaluator/local_evaluator.py`

Context. This document explains, with file and line references, how the multi-turn shopping-search competition's simulated customer produces its messages from the JSONL data (see `docs/competition_specification.md`). No code changes were made; this is a read-only investigation summary.

The short version

It is a **rule-based, template-filling state machine** — no LLM call and no free-text NLP. Every word the “customer” says is a verbatim substring pulled from the real product's own catalog fields.

Where the logic lives

All of it is in one file, owned by the evaluator (not the participant's own agent code in `src/`):

`evaluator/local_evaluator.py` — intent-card synthesis, scenario behavior, the turn-1 message, and every later customer reply.

Function	Lines	Role
<code>intent_card(product, limit=180)</code>	52–71	Builds the hidden intent card from a catalog record
<code>behavior_for(scenario, card, rng)</code>	74–87	Builds scenario-specific behavior (Intent Override only, currently)
<code>materialize_hidden_fields(sample, products)</code>	204–213	Synthesizes <code>intent_card</code> /behavior at eval time
<code>initial_message(sample, category, disclosed)</code>	154–163	Turn-1 customer message, scenario-dependent
<code>customer_reply(sample, ask_attribute, disclosed, boundary_used)</code>	166–185	Turn-2+ customer message, driven by the agent's <code>ask_attribute</code>
<code>classify_constraint(value)</code>	137–151	Maps a constraint phrase to one of 10 allowed attributes
<code>evaluate(...)</code>	216–295	Per-session driver loop tying it all together

The two JSONL layers

- `data/catalog.jsonl` — 50,000 real products: title, features, description, price, categories, details, `average_rating`, `rating_number`, store.
- `data/public_set.jsonl` / `hard_set.jsonl` — session definitions only: `sample_id`, `scenario_type`, `ground_truth.parent_asin`, `user_profile`. **No dialogue or intent card is shipped** — `data/README.md` states this explicitly.

The pipeline, step by step

- 1. Lookup.** `materialize_hidden_fields()` (line 204) resolves the session's `ground_truth.parent_asin` against `catalog.jsonl`.
- 2. Intent-card synthesis.** `intent_card()` (line 52) regex-mines that product's features/details/title/price — a material regex, a color regex, a price pattern — into 2 `hard_constraints` + up to 2 `soft_preferences`. All verbatim text from the listing; nothing is rewritten or LLM-generated.
- 3. Scenario behavior.** `behavior_for()` (line 74) — for Intent Override sessions only, deterministically (seeded from `sample_id`) rolls the override to turn 3 or 4 and prebuilds the exact reversal sentence.
- 4. Turn 1.** `initial_message()` (line 154) — a fixed template keyed on `scenario_type`: **Buying** discloses `hard_constraints[0]` immediately; **Intent Override** opens with the preference that will later flip; **Browsing/Boundary** reveal nothing (“still exploring”).
- 5. Every later turn.** `customer_reply()` (line 166) is driven **only** by the agent's structured `ask_attribute` field — not by parsing the agent's prose message (per `docs/competition_specification.md`, line 61):
 - **Boundary**, first attribute asked: “I don't have a preference for X; please use your judgment” (fires once).
 - **ask_attribute is null**: generic “ask me about one specific attribute” filler.
 - **Otherwise**: `classify_constraint()` (line 137) buckets remaining undisclosed constraints into 10 attribute types, returns up to 2 matches, marks them disclosed; no match → “I don't have an additional preference for X.”
 - **Intent Override**: on the pre-rolled turn, the reversal sentence force-fires regardless of what was asked, and hits don't count until it has.
- 6. Driver loop.** `evaluate()` (line 216) ties it together each turn — call the agent, check for a hit, else call the override injection or `customer_reply()` to produce the next `user_message`.

Why this matters

The “customer” never invents information — it reveals pieces of a real Amazon listing on a fixed schedule (a turn-1 template, then attribute-gated disclosure). That's also why the agent's `ask_attribute` field matters so much: it is literally the only key that unlocks new information after turn 1.

Source references

`evaluator/local_evaluator.py`: lines 27–37, 40–45, 48–49, 52–71, 74–87, 137–151, 154–163, 166–185, 204–213, 216–295

`docs/competition_specification.md`: lines 7, 19, 25–28, 32–40, 60–61

`data/README.md`: lines 5, 7

`IMPLEMENTATION.md`: lines 165, 228–229, 461, 549–554, 776, 1073–1102